

Tuples

31 July 2026 11:04

A tuple is an ordered sequence of elements of different data types, such as integer, float, string, list, or even a tuple. Elements of a tuple are enclosed in parenthesis and are separated by commas. Like list and string, elements of a tuple can be accessed using index values starting from 0.

```
>>> t1 = (1,2,3,4,5)
>>> t1
(1, 2, 3, 4, 5)
>>> type(t1)
<class 'tuple'>
>>>
```

#t2 is the tuple of mixed data types

```
>>> t2 = ('Economics', 99, 'Maths', 100, 'Accountancy', 95.7)
>>> t2
('Economics', 99, 'Maths', 100, 'Accountancy', 95.7)
>>> type(t2)
<class 'tuple'>
```

#t3 is the tuple with tuple as an element

```
>>> t3 = ('Unit1', (20, 21, 24, 19), 'Unit-2', (19, 23, 21, 18))
>>> t3
('Unit1', (20, 21, 24, 19), 'Unit-2', (19, 23, 21, 18))
```

If there is only a single element in a tuple, then the element should be followed by a comma. If we assign the value without, it is treated as integer. It should be noted that sequence without parentheses is treated as tuple by default.

#Incorrect way to define single element in the tuple.

```
>>> t4 = (20)
>>> type(t4)
<class 'int'>
```

#Correct way to define single element tuple.

```
>>> t4 = (20,)
>>> type(t4)
<class 'tuple'>
>>>
```

#A sequence without parenthesis is treated as tuple by default.

```
>>> nums = 10, 20, 30, 40, 50
>>> type(nums)
<class 'tuple'>
>>> nums
```

```
(10, 20, 30, 40, 50)
```

```
>>>
```

Accessing elements in a tuple

Elements of a tuple can be accessed in the same way as a list or string using indexing and slicing.

0	1	2	3	4
10	20	30	40	50
-5	-4	-3	-2	-1

```
>>> nums[:-1] ← values before '-' index.  
(10, 20, 30, 40)
```

#reverse the elements of tuple

```
>>> nums[::-1]  
(50, 40, 30, 20, 10)
```

For wrong index it generates index error:

```
>>> nums[10]  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    nums[10]  
~~~~^
```

IndexError: tuple index out of range

```
>>> nums[-10]  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    nums[-10]  
~~~~^
```

IndexError: tuple index out of range

Tuple is immutable: Tuple is an immutable data type. It means that the element of a tuple cannot be changed after it has been created. An attempt to do this would lead to an error.

```
>>> nums  
(10, 20, 30, 40, 50)  
>>> nums[3] = 80  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
    nums[3] = 80  
~~~~^  
TypeError: 'tuple' object does not support item assignment  
>>>
```

However, an element of a tuple may be of mutable type.

```
>>> num2 = (10, 20, 30, 40, [5, 15])  
>>> num2[4][1] = 30
```

```

>>> num2
(10, 20, 30, 40, [5, 30])
>>> num2[4].append(90)
>>> num2
(10, 20, 30, 40, [5, 30, 90])
>>>

```

Tuple Operations

Concatenation: Python allows us to join tuples using concatenation operator depicted by a symbol plus(+). We can also create a new tuple which contains the result of this concatenation operation.

```

>>> t1
(1, 2, 3, 4, 5)
>>> t2
('Economics', 99, 'Maths', 100, 'Accountancy', 95.7)
>>> t1 = t1 + t2
>>> t1
(1, 2, 3, 4, 5, 'Economics', 99, 'Maths', 100, 'Accountancy', 95.7)
>>> t2
('Economics', 99, 'Maths', 100, 'Accountancy', 95.7)
>>>

```

Concatenation operator can also be used for extending an existing tuple. When we extend the tuple using concatenation, a new tuple is created. And assign to the variable or object.

Repetition: Repetition operation is depicted by the symbol * It is used to repeat elements of a tuple. We can repeat tuple elements. The repetition operator requires the first operand to be a tuple or and the 2nd operand to be an integer only.

```

>>> t6 = ('Hello', 'Python')
>>> t6 * 5
('Hello', 'Python', 'Hello', 'Python', 'Hello', 'Python', 'Hello', 'Python',
'Hello', 'Python')
>>> t6
('Hello', 'Python')
>>> t6 = t6 * 5
>>> t6
('Hello', 'Python', 'Hello', 'Python', 'Hello', 'Python', 'Hello', 'Python',
'Hello', 'Python')
>>>

```

Membership: The **in** operator checks if the element is present in the tuple and returns True else it returns False.

```
>>> color = ('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta')
>>> color
('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta')
>>> 'Blue' in color
True
>>> 'Yellow' in color
False
```

The **not in** operator returns True if the element is not present in the tuple, else it returns False.

```
>>> color
('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta')
>>> 'Yellow' not in color
True
>>> 'Violet' not in color
False
```

Slicing: Like string and list slicing can be applied to tuples also.

```
>>> color
('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta')
>>> color[2:5]
('Blue', 'Brown', 'Violet')
>>> color[2:]
('Blue', 'Brown', 'Violet', 'Magenta')
>>> color[: ]
('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta')
>>> color[:: -1]
('Magenta', 'Violet', 'Brown', 'Blue', 'Green', 'Red')
>>> color[:3]
('Red', 'Green', 'Blue')
>>>
```

Tuple methods and built in functions

Python provides many functions to work on tuples. The commonly used tuple methods and built in functions are as follows:

Built-in functions and methods for tuples

Method	Description	Example
len()	Returns the length or number of elements of the tuple passed as the argument.	<pre>>>> color ('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta') >>> len(color) 6</pre>

tuple()	Create an empty tuple if no argument is passed. Creates a tuple if a sequence is passed as an argument.	<pre> >>> t10 = tuple() >>> t10 () >>> t11 = tuple('aeioAEIOU') >>> t11 ('a', 'e', 'i', 'o', 'A', 'E', 'I', 'O', 'U') >>> li1 = [2, 4, 6, 8] >>> even = tuple(li1) >>> even (2, 4, 6, 8) >>> </pre>
count()	Returns the number of times the given element appears in a tuple.	<pre> >>> tup1 = (10, 20, 30, 10, 20, 40, 50, 20) >>> tup1.count(20) 3 >>> tup1.count(90) 0 </pre>
index()	Returns the index of the first occurrence of the element in the given tuple.	<pre> >>> tup1 (10, 20, 30, 10, 20, 40, 50, 20) >>> tup1.index(20) 1 >>> tup1.index(20, 1) 1 >>> tup1.index(20, 2) 4 >>> tup1.index(60, 2) Traceback (most recent call last): File "<stdin>", line 1, in <module> tup1.index(60, 2) ~~~~~^~~~~^ ValueError: tuple.index(x): x not in tuple </pre>
sorted()	Takes element in the tuple and returns a new sorted list. It should be noted that sorted() does not make any change to the original tuple.	<pre> >>> tup1 (10, 20, 30, 10, 20, 40, 50, 20) >>> s1 = sorted(tup1) >>> s1 [10, 10, 20, 20, 20, 30, 40, 50] >>> color ('Red', 'Green', 'Blue', 'Brown', 'Violet', 'Magenta') >>> scl = sorted(color) >>> scl ['Blue', 'Brown', 'Green', </pre>

		'Magenta', 'Red', 'Violet'] >>>
min()	Return the minimum or smallest element of the tuple.	>>> tup1 (10, 20, 30, 10, 20, 40, 50, 20)
max()	Returns maximum or largest element of the tuple.	>>> min(tup1) 10 >>> max(tup1) 50
sum()	Returns sum of the elements of the tuple.	>>> sum(tup1) 200

Tuple Assignment

Assignment of tuple is a useful feature in Python. It allows a tuple of variables on the left side of the assignment operator to be assigned respective values from a tuple on the right side. The number of variables on the left should be same as the number of elements in the tuple.

```
>>> (num1, num2) = (10, 20)
>>> num2
20
>>> num1
10
```

```
>>> record = ('Shiv', 50, 'CS')
>>> (name, marks, subject) = record
>>> name
'Shiv'
>>> marks
50
>>> subject
'CS'
```

```
>>> (a, b, c) = (10, 5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    (a, b, c) = (10, 5)
    ^^^^^^^^^^^
```

ValueError: not enough values to unpack (expected 3, got 2)

If there is an expression on the right side, then first that expression is evaluated and finally the result is assigned to the tuple.

```
>>> (n1, n2) = (10+5, 4 * 4.3)
>>> n1
15
>>> n2
17.2
>>>
```

Nested Tuples

A tuple inside another tuple is called a nested tuple.

```
st = ((101, 'Aman', 98),
      (102, 'Dipak', 82),
      (103, 'Raj', 76),
      (104, 'Shiv', 99),
      (105, 'Amit', 88))
print(f'\tRoll No.\tName\tMark')
for record in st:
    print(f'\t{record[0]}\t\t{record[1]}\t{record[2]}')
```

Output:

Roll No.	Name	Mark
101	Aman	98
102	Dipak	82
103	Raj	76
104	Shiv	99
105	Amit	88

Homework:

1. Write a program to swap two numbers without using a temporary variable.
2. Write a program to input N numbers from the users. Store these numbers in a tuple. Print the maximum, minimum and sum of this tuple.
3. Write a program to read email IDS of N number of students and store them in a tuple. Create two new tuples, one to store the usernames from the email IDS, and second choose store domain names from the email ID. Print all three tuples at the end of the program.
4. Write a program to input names of N students and store them in a tuple. Also input a name from the user and find if this student is present in the tuple or not.